

Communications entre threads, l'exemple ProdCons (v2)

Fabrice.Kordon@lip6.fr



Objectif de la seconde version

2

Rendre atomique le test et la modification

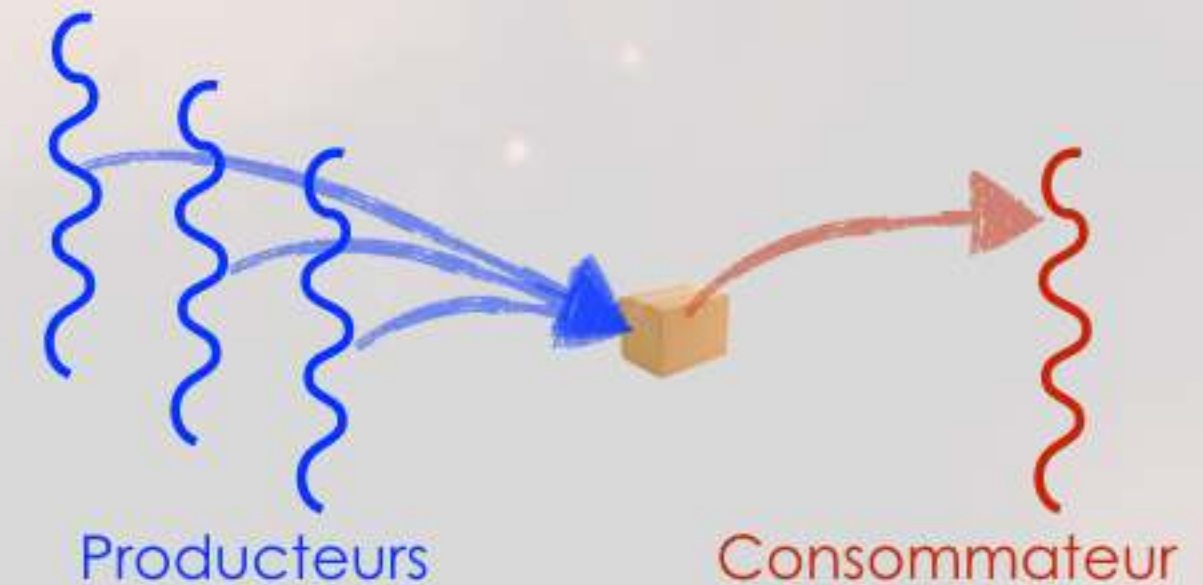
 ajouteValeur()

▶ Fonctionne ou lève l'exception Recommencer

 retireValeur()

▶ Fonctionne ou lève l'exception Recommencer

 Si l'exception Recommencer est levée, il faut refaire l'opération



Source d'inspiration...

 L'opération TestAndSet

▶ Écriture atomique

 Son extension, CompareAndSwap

▶ Comparaison à une valeur donnée et échange avec une autre valeur en cas d'égalité

La nouvelle classe BoiteMessage

3

```
@SuppressWarnings("serial")// éviter un warning dans Recommencer
```

```
class Recommencer extends Exception {  
    Recommencer (String m) {  
        super (m);  
    }  
}
```

La nouvelle classe BoiteMessage

3

```
class BoiteMessage {
    boolean plein;
    int valeur;

    BoiteMessage() {
        plein = false;
    }

    synchronized public void ajouteValeur (int val) throws Recommencer {
        if (plein) {
            throw new Recommencer("Boîte déjà pleine");
        }
        valeur = val;
        plein = true;
    }

    synchronized public int retireValeur () throws Recommencer {
        if (! plein) {
            throw new Recommencer("Boîte vide");
        }
        plein = false;
        return valeur;
    }
}
```

```
@SuppressWarnings("serial")// éviter un warning dans Recommencer

class Recommencer extends Exception {
    Recommencer (String m) {
        super (m);
    }
}
```

La nouvelle classe Consommateur

4

```
class Consommateur implements Runnable {
    private BoiteMessage b;
    private boolean actif = true;

    Consommateur (BoiteMessage b) {
        this.b = b;
    }

    private void trace (String m) {
        System.out.println ("Serveur : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        this.trace ("initialisé");
        while (actif) {
            try {
                int val = b.retireValeur();
                this.trace ("retire " + val);
            }
            catch (Recommencer e) {
                this.trace("échec, " + e);
                try {
                    Thread.sleep(1000); // pour ralentir
                }
                catch (Exception e2) {
                    this.trace(e2 + "Ne doit pas arriver");
                }
            }
        }
        this.trace ("terminé");
    }
}
```

La nouvelle classe Producteur

5

```
class Producteur implements Runnable {
    static final public int cstMaxProd = 3; // constante
    private BoiteMessage b;
    private int monNumero;
    private boolean actif = true;

    Producteur (int i, BoiteMessage b) {
        monNumero = i;
        this.b = b;
    }
    private void trace (String m) {
        String indent = String.format("%" + monNumero* 2 + "s", "");
        System.out.println (indent + "Client " + monNumero + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }
    public void run() {
        this.trace ("initialisé");
        while (actif) {
            try {
                actif = false;
                b.ajouteValeur(monNumero);
            }
            catch (Recommencer e) {
                this.trace("On recommence, " + e);
                actif = true;
                try {
                    Thread.sleep(1000); // pour ralentir
                }
                catch (Exception e2) {
                    this.trace(e2 + "Ne doit pas arriver");
                }
            }
        }
        this.trace ("terminé");
    }
}
```

La classe principale (rien de changé)

6

```
public class ProdCons2 {  
  
    public static void main(String []args) {  
  
        BoiteMessage boite = new BoiteMessage();  
  
        Thread cons = new Thread(new Consommateur (boite));  
        Thread prods[] = new Thread [Producteur.cstMaxProd];  
  
        cons.start(); // démarrer le serveur  
  
        for (int i = 0; i < prods.length ; i++) {  
            prods[i] = new Thread(new Producteur (i+1, boite));  
            prods[i].start(); // démarrer un client  
        }  
    }  
}
```

La classe principale (rien de changé)

6

```
public class ProdCons2 {
```

```
$ java ProdCons2
```

```
Serveur : initialisé
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
Client 1 : initialisé
```

```
Client 2 : initialisé
```

```
Client 3 : initialisé
```

```
Client 1 : terminé
```

```
Client 2 : On recommence, BoiteMessage$Recommencer: Boîte déjà pleine
```

```
Client 3 : On recommence, BoiteMessage$Recommencer: Boîte déjà pleine
```

```
Serveur : retire 1
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
Client 2 : On recommence, BoiteMessage$Recommencer: Boîte déjà pleine
```

```
Client 3 : terminé
```

```
Serveur : retire 3
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
Client 2 : terminé
```

```
Serveur : retire 2
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
^C
```

La classe principale (rien de changé)

6

```
public class ProdCons2 {
```

```
$ java ProdCons2
```

```
Serveur : initialisé
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
    Client 3 : initialisé
```

```
    Client 2 : initialisé
```

```
    Client 1 : initialisé
```

```
    Client 3 : terminé
```

```
    Client 1 : On recommence, BoiteMessage$Recommencer: Boîte déjà pleine
```

```
    Client 2 : On recommence, BoiteMessage$Recommencer: Boîte déjà pleine
```

```
Serveur : retire 3
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
    Client 1 : On recommence, BoiteMessage$Recommencer: Boîte déjà pleine
```

```
    Client 2 : terminé
```

```
Serveur : retire 2
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
    Client 1 : terminé
```

```
Serveur : retire 1
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
Serveur : échec, BoiteMessage$Recommencer: Boîte vide
```

```
^C
```

Gérer la terminaison du Consommateur?

7

```
public void run() {  
    this.trace ("initialisé");  
    for (int i = 0 ; i < Producteur.cstMaxProd ; i++) {  
        try {  
            int val = b.retireValeur();  
            this.trace ("retire " + val);  
        }  
        catch (Recommencer e) {  
            i--; // ne pas compter cet échec  
            this.trace("échec, " + e);  
            try {  
                Thread.sleep(1000); // pour ralentir  
            }  
            catch (Exception e2) {  
                this.trace(e2 + "Ne doit pas arriver");  
            }  
        }  
    }  
    this.trace ("terminé");  
}
```

Gérer la terminaison du Consommateur?

7

```
public void run() {
```

```
$ java ProdCons3
```

```
Serveur : initialisé
```

```
Serveur : échec, Recommencer: Boîte vide
```

```
Client 2 : initialisé
```

```
Client 1 : initialisé
```

```
Client 3 : initialisé
```

```
Client 2 : terminé
```

```
Client 1 : On recommence, Recommencer: Boîte déjà pleine
```

```
Client 3 : On recommence, Recommencer: Boîte déjà pleine
```

```
Serveur : retire 2
```

```
Serveur : échec, Recommencer: Boîte vide
```

```
Client 3 : terminé
```

```
Client 1 : On recommence, Recommencer: Boîte déjà pleine
```

```
Serveur : retire 3
```

```
Serveur : échec, Recommencer: Boîte vide
```

```
Client 1 : terminé
```

```
Serveur : retire 1
```

```
Serveur : terminé
```

Ici on a de la chance...

La terminaison est en général délicate

Autres approches pour la terminaison (plus parallèles 😊)

8

Programme principal

- 🔧 Initialiser une variable partagée avec le nombre de clients

Les Clients

- 🔧 Avant de se terminer, décrémenter cette variable partagée

Le serveur

- 🔧 Consulter cette variable partagée et se terminer si elle vaut 0

Facile en multi-threadé...

Car on peut communiquer
par variables partagées